Project 02 — Document & Image Analysis on SGLang

GPU server · vision-language · structured extraction at volume

Contents

Project 02 — Document & Image Analysis (SGLang)

Read `00-shared-requirements.md` first. Everything in it applies here.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **SGLang serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §6 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a "Say this" line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest `source` badge — except Improvement, which is never seeded.

8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §7. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §7a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §8 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §12 of this document plus the shared deliverables checklist.

---

1. Brief

Build an **automated document understanding service**: upload an invoice, a claim form, an ID card or a shipping manifest, and get back **validated structured JSON** plus a visual overlay showing where on the page each field was read from.

Not a chat interface over images. A pipeline that a finance or operations team runs thousands of documents through daily, whose output feeds a downstream system, and whose errors cost money.

Two properties make it credible rather than a toy:

> **Every extracted field carries a bounding box and a confidence score, and the service refuses rather than guesses below a threshold.**

A field with no provenance cannot be checked by the human reviewing it. A pipeline that always returns a value silently converts a low-confidence guess into a downstream error — which is exactly how document AI projects fail in production.

Target user: a company doing high-volume manual data entry from documents — accounts payable, insurance claims intake, logistics, KYC onboarding.

---

2. Why SGLang for this workload

SGLang's advantages are unusually well matched here, and both are about *repetition*:

- **RadixAttention.** SGLang keeps a radix tree of KV cache across requests, so any shared prefix is reused automatically — not just an exact-match prefix, but any common branch. Document extraction is the ideal case: every request for a given document type shares the same long system prompt and the same schema instructions. Processing ten thousand invoices means that prefix is prefilled once, not ten thousand times. On a long extraction prompt this is the dominant cost saving in the whole system, and it is the number to put in front of the client.
- **Structured output with constrained decoding.** SGLang can enforce a JSON schema or regex during generation, at the decoding level. The model *cannot* emit invalid JSON — it is not asked politely and then parsed hopefully. For a pipeline feeding a downstream system, this removes an entire class of production failure, and it is faster than generate-then-retry.
- **Its frontend language** expresses multi-step control flow — extract, then branch on document type, then extract type-specific fields — inside the runtime rather than across multiple round trips.

Compare honestly in the README: vLLM also has prefix caching, but SGLang's radix tree handles *branching* prefixes better, which is what a multi-document-type workload actually produces. Say where the advantage is real and where it is marginal.

---

3. Model and compression

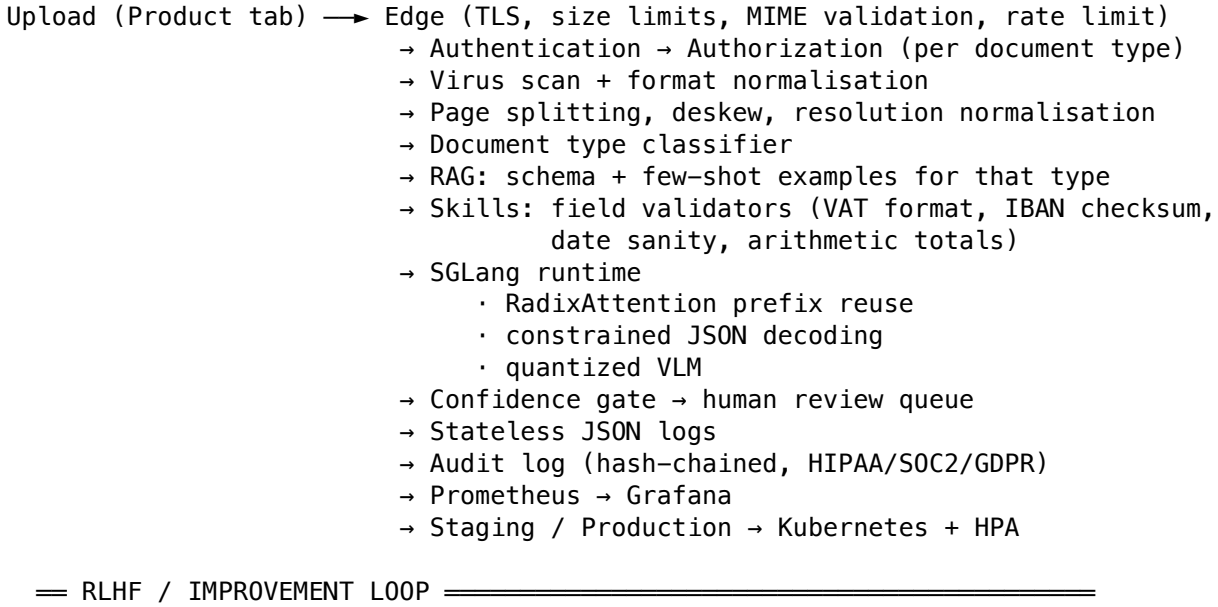
Component	Choice
Vision-language model	`Qwen2.5-VL-7B-Instruct` (strong document OCR and grounding)
Alternative	`InternVL2-8B`
Quantization	W4A16 (GPTQ/AWQ) or FP8 on Hopper/Ada
Embeddings	`bge-m3` for the document-type classifier and few-shot example retrieval

Compression spec - Quantize with a **document-domain calibration set** — real invoices, forms and scans, not generic web text. Quantization scales are fitted to the tokens the model sees; calibrating on the wrong distribution leaves accuracy on the table. Vision-language models are particularly sensitive here because the visual token distribution is nothing like web text. - Record weights before and after and state what the freed memory buys: longer document context and more pages in flight per GPU.

Quality gate — field-level, not document-level - Per-field precision and recall against a labelled holdout set, gated as a delta against the uncompressed baseline. - **Bounding box IoU** against ground truth — a correct value attributed to the wrong region is still a failure, because the human reviewer cannot verify it. - Schema conformance rate (should be 100% with constrained decoding — if it is not, the constraint is misconfigured). - **Calibration check:** confidence scores must be meaningfully ordered. A model that reports 0.9 on wrong answers is worse than one that reports 0.5 on everything, because the refusal threshold stops working.

---

4. Architecture



Points worth building deliberately:

- **Validators are code, not prompts.** An IBAN checksum, a VAT format, whether line items sum to the stated total — these are deterministic and must be computed, not asked of a model. When a validator

disagrees with the model, that is the highest value signal in the system: route it to human review.

- **The human review queue is part of the product, not an admission of failure.** Show it. A client evaluating this is comparing against 100% manual entry; a system that handles 85% automatically and routes 15% with the uncertain fields highlighted is an enormous win, and being honest about the 15% is more persuasive than claiming 100%.
- **Batch endpoint** for the real workload. Interactive upload is the demo; the business case is ten thousand documents overnight, where RadixAttention's prefix reuse compounds.

---

5. Frontend

Product tab — split view: document on the left with **bounding box overlays**, extracted JSON on the right. Hovering a JSON field highlights its box on the page and vice versa. Low-confidence fields are visually flagged and editable, and a correction is recorded as training signal.

That linked hover is the single most convincing interaction in this project. It turns "the model said 1,240.50" into "the model read 1,240.50 from *here*", which is the difference between a claim and evidence.

Configuration: document types, per-type JSON schema, confidence thresholds, validation rules, review routing.

Architecture tab — 3D graph. Logos: SGLang, NVIDIA, Kubernetes, Prometheus, Grafana, PostgreSQL, Qwen. The nodes needing the strongest copy are **RadixAttention**, **constrained decoding**, **the confidence gate** and **the validators**.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §6 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

---

6. Architecture graph: node inventory

These are the stage cards for this project's Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent's card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a "Say this" line on each.

#	Stage card	Colour role	Expands into
0	**User**	neutral	— (uploads a document)
1	**Upload Console**	teal	Drag-and-drop, page preview, extraction review, schema editor
2	**Edge & Security**	amber	TLS, rate limiting, authentication, authorization, file-type and size validation, malicious-document scanning
3	**Document Pipeline**	violet	Page splitting, OCR, layout detection, region ordering, image normalisation

#	Stage card	Colour role	Expands into
4	**Context & Schema**	violet	Target schema, few-shot exemplars, retrieval of prior extractions, prompt compiler, injection defence on document text
5	**SGLang Runtime**	orange	**Qwen2.5-VL-7B-Instruct (INT4/FP8)**, RadixAttention prefix reuse, structured/constrained decoding, continuous batching, paged KV cache
6	**Validation**	violet	Schema conformance, confidence gating, human-review routing
7	**Database**	blue	PostgreSQL + pgvector, extraction store, document registry
8	**Monitoring & Audit**	magenta	Prometheus, Grafana, hash-chained audit log, structured logging, alerting
9	**Improvement Loop**	green	Field-level correction capture, A/B extraction comparison, preference dataset, extraction scoring, DPO fine-tune, promotion gate `lm_eval`, `GuideLLM`
10	**Platform**	grey	Kubernetes, HPA on queue depth, staging production

The card carrying the argument is **SGLang Runtime**, and specifically **RadixAttention**: every document in a batch shares the same schema prompt and the same few-shot exemplars, so the shared prefix is enormous and prefix reuse is worth far more here than in a chat workload. Make the detail panel say what fraction of the prompt is shared.

---

7. The improvement loop for this project

`00-shared-requirements.md` §5a applies in full. What is specific here:

Document extraction has the best feedback signal of any project in this pack, because **the correction is the label**:

- **Field-level correction** — the reviewer fixes one extracted value in the review UI. That edit is simultaneously a bug report, a training example and an accuracy measurement. Capture it as `(document region, extracted, corrected)`.
- **Preference** — the same document extracted twice under different prompts or sampling settings, shown side by side with the differing fields highlighted.
- **Rating** — accept / reject on a whole extraction, for volume.

Rule specific to this domain: **store the page region, not just the text.** A correction without the region it came from cannot be used to train a vision-language model, and reconstructing it later is unreliable once

the document pipeline changes.

7a. LoRA adapters: where they earn their place here

This is the project where LoRA pays best, so build it and demonstrate it.

Document extraction is naturally multi-tenant and naturally narrow: every client has their own document types, their own schema and their own conventions about what "invoice date" means. That is precisely the shape a small adapter fits, and precisely the shape a single general model handles poorly.

Test this: one adapter per document type, served from one base model.

Approach	GPU cost for 15 document types	Accuracy on a narrow type
One general model, prompt-only	1×	worst — the model hedges across conventions it has seen
Fine-tuned model per type	15×	best, and unaffordable
Base model + LoRA per type	**1×**	close to per-type fine-tuning

SGLang serves multiple adapters against one base model, so the routing decision is "which adapter for this document type", made at request time. Build that routing as a visible node in the architecture graph — "one GPU, fifteen specialisms" is the clearest cost story in this entire pack.

Where the adapters come from. The improvement loop already produces them: field-level corrections on invoices train the invoice adapter, not a global model. That containment is a feature — a bad batch of corrections on one document type cannot degrade extraction on the other fourteen. Say that explicitly; it is the answer to "what if the feedback data is bad".

Measure: per-adapter extraction accuracy against the shared base, adapter swap latency, and prefix-cache hit rate with adapters active — check that adapter switching has not quietly destroyed prefix sharing, which would leave accuracy untouched and multiply the bill.

---

8. The release gate for this project

`00-shared-requirements.md` §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	`lm_eval` + held-out extraction suite scored field-by-field against ground truth	**Per-field exact-match and schema-conformance rate.** An aggregate accuracy hides the case where one critical field degraded and forty trivial ones improved
Performance	Is it still fast?	`GuideLLM` at a realistic document arrival rate, with realistic prompt lengths	**Prefix-cache hit rate is the cost metric.** A change that breaks prefix sharing can leave quality untouched and multiply the bill

Benchmark with **real page counts and real prompt lengths**. A benchmark run with a 128-token prompt measures a workload this system never serves.

9. Monitoring and KPIs

Section	Metrics
Quality	Per-field precision/recall, bounding box IoU, schema conformance, straight-through processing rate, human correction rate by field
Performance	Latency per page p50/p95/p99, pages/sec, **RadixAttention cache hit rate**, KV cache utilisation, batch occupancy, GPU utilisation
Security	Malicious upload attempts, **prompt injection embedded in document text** (a real and under-appreciated attack — text inside an uploaded PDF instructing the model), auth failures, 4xx/5xx
Audit	Every extraction: who, which document, which model version, which config version, what was returned, who corrected it
Improvement (RLHF)	Field-level correction rate (per field), extraction approval rate, challenger win rate, preference pairs awaiting the next training run

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

The cost metric that sells the project: cost per document processed, next to the loaded cost of manual entry. Include the human review cost honestly — the number is still overwhelming, and an honest number survives scrutiny.

Watch the RadixAttention hit rate. A drop means prefixes stopped being shared — usually because someone put a per-request value (a timestamp, a document ID) at the front of the prompt. That single mistake can multiply serving cost several-fold and is invisible without this metric.

10. Security, audit and compliance

- **Documents are the attack surface.** Text inside an uploaded PDF instructing the model to ignore its schema is a working attack. The system prompt must state that document content is data, never instructions, and the pipeline must detect and count injection attempts found in extracted text.
- Uploaded documents routinely contain special-category data — health claims, identity documents. Encrypt with a dedicated key, define retention, support erasure, and keep source documents out of any general-purpose data store.
- Audit every extraction with model version and config version. When a downstream system acts on a wrong value six months later, this is the only way to reconstruct what happened.
- Per-document-type authorization: a user cleared for invoices should not be able to submit medical claims.

11. Deployment

Kubernetes with a GPU node pool. VLMs need more memory than a text model of the same parameter count — image tokens are numerous — so size for that and verify before committing to an instance type.

Two serving paths worth separating: an interactive endpoint tuned for latency, and a batch endpoint tuned for throughput with a much larger batch size. They have opposite optimal settings, and forcing one configuration to serve both leaves a large amount of throughput unused.

---

12. Deliverables

- Upload structured JSON with bounding boxes and confidence
- Batch processing endpoint
- Human review queue with correction capture
- Quantization pipeline with a field-level quality gate
- All three frontend tabs
- Grafana dashboards including RadixAttention hit rate
- Helm charts and infrastructure as code
- Measured throughput and cost per document

Acceptance: 1. Field-level precision above the agreed threshold on a labelled holdout set 2. 100% schema conformance via constrained decoding 3. Injection text embedded in an uploaded document is detected, counted, and does not alter extraction behaviour 4. RadixAttention hit rate demonstrably above 60% on a homogeneous batch 5. Architecture tab and guided tour run with the backend stopped

---

13. Out of scope

Handwriting recognition beyond what the base model does. Training a custom detector. Non-Latin scripts, beyond confirming nothing crashes. Direct ERP integration — expose an API and stop there.